

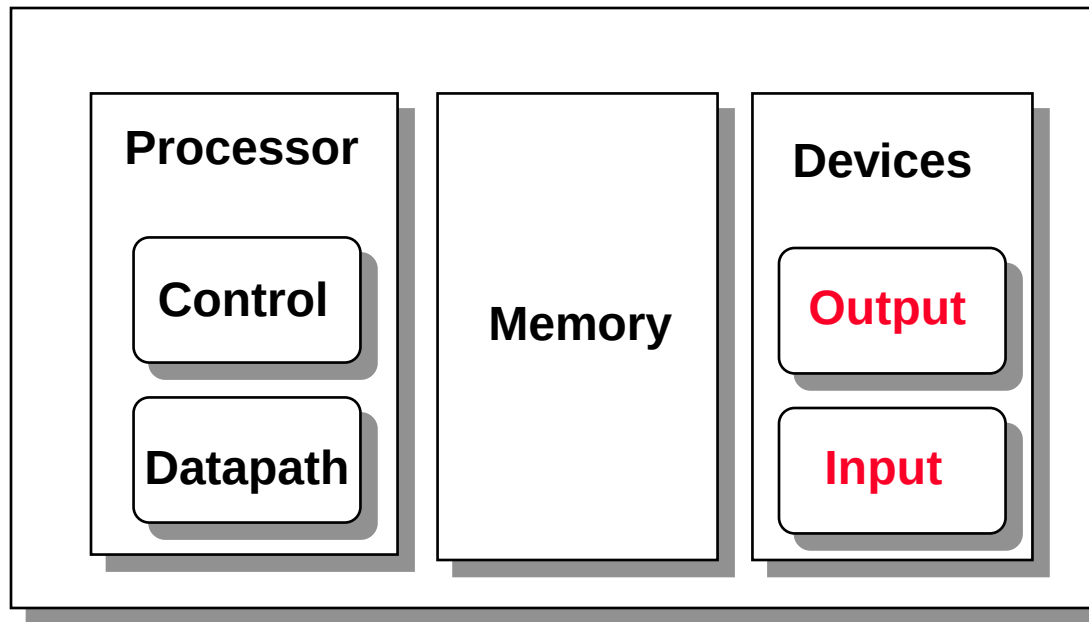
---

# Chapter 6: I/O System

Ngo Lam Trung

[with materials from *Computer Organization and Design, 4<sup>th</sup> Edition*,  
Patterson & Hennessy, © 2008, MK  
and M.J. Irwin's presentation, PSU 2008]

# Review: Major Components of a Computer



❑ Input + Output = I/O system

❑ Hard disk

❑ Network

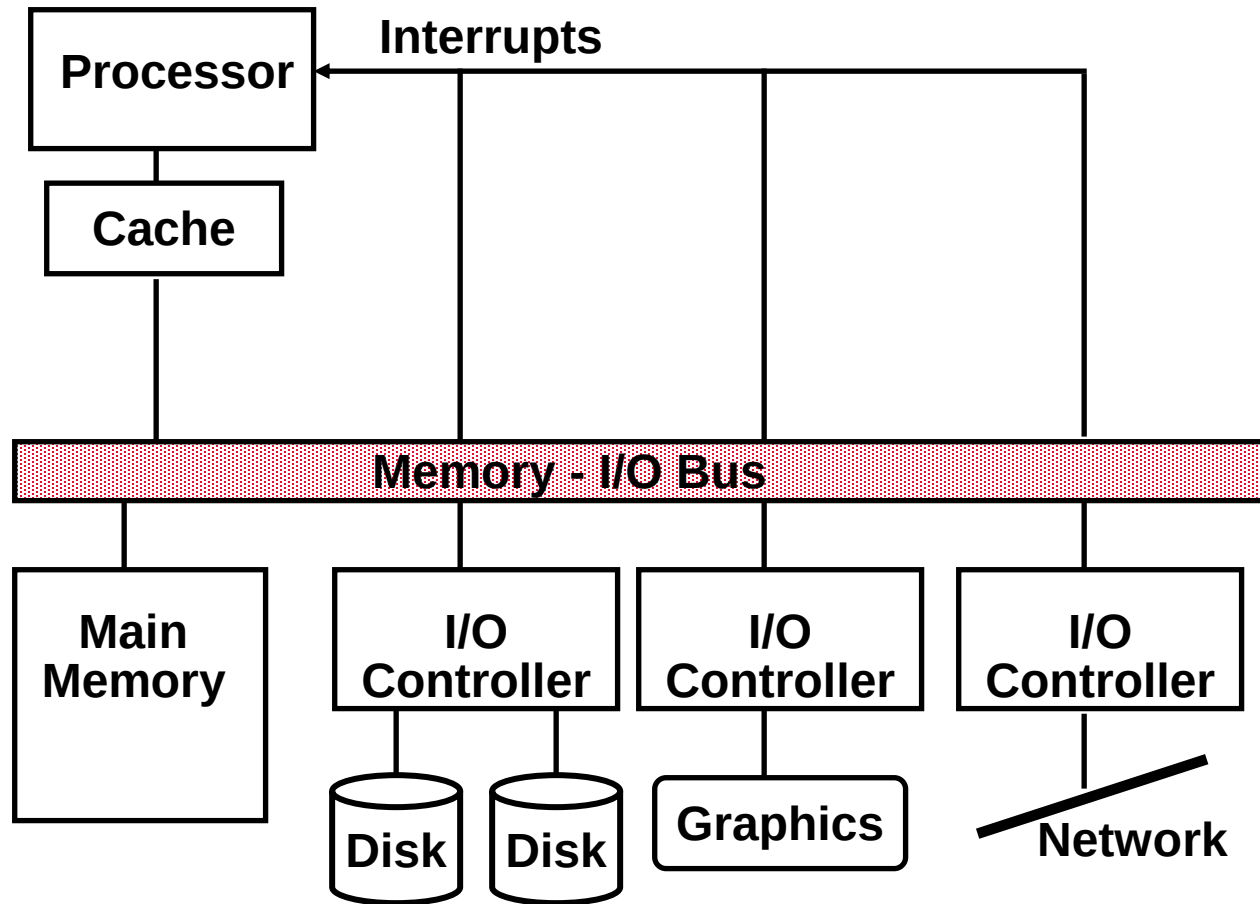
Anything else?

❑ USB drive

# Important metrics for an I/O system

- ❑ Performance
- ❑ Expandability
- ❑ Dependability
- ❑ Cost, size, weight
- ❑ Security
  
- ❑ What is the most important?

# A Typical I/O System



# Input and Output Devices

- ❑ I/O devices are incredibly diverse with respect to
  - ❑ Behavior – input, output or storage
  - ❑ Partner – human or machine
  - ❑ Data rate – the peak rate at which data can be transferred between the I/O device and the main memory or processor

| Device           | Behavior        | Partner | Data rate (Mb/s)    |
|------------------|-----------------|---------|---------------------|
| Keyboard         | input           | human   | 0.0001              |
| Mouse            | input           | human   | 0.0038              |
| Laser printer    | output          | human   | 3.2000              |
| Magnetic disk    | storage         | machine | 800.0000-3000.0000  |
| Graphics display | output          | human   | 800.0000-8000.0000  |
| Network/LAN      | input or output | machine | 100.0000-10000.0000 |

# I/O Performance Measures

---

- ❑ **I/O bandwidth** (throughput) – amount of information that can be input/output and communicated across an interconnect between the processor/memory and I/O device per unit time
  1. How much data can we move through the system in a certain time?
  2. How many I/O operations can we do per unit time?
- ❑ **I/O response time** (latency) – the total elapsed time to accomplish an input or output operation
- ❑ Many applications require *both* high throughput and short response times

# Dependability: Reliability and Availability

- ❑ Mean Time To Failure (MTTF): average time of normal operation between two consecutive failure
- ❑ Mean Time To Repair (MTTR): average time of service interruption when failure occurs
- ❑ Reliability: measured by MTTF
- ❑ Availability:

$$\text{Availability} = \frac{\text{MTTF}}{(\text{MTTF} + \text{MTTR})}$$

- ❑ Example: Seagate ST33000655SS MTTF = 1400000 hours @25°C

# Physical connection

---

- ❑ Processor
- ❑ Memory
- ❑ I/O devices
  
- ❑ How to connect them physically?

# I/O System Interconnect Issues

- ❑ A **bus** is a shared communication link (a single set of wires used to connect multiple subsystems) that needs to support a range of devices with widely varying latencies and data transfer rates
  - ❑ Advantages
    - Versatile – new devices can be added easily and can be moved between computer systems that use the same bus standard
    - Low cost – a single set of wires is shared in multiple ways
  - ❑ Disadvantages
    - Creates a communication bottleneck – bus **bandwidth** limits the maximum I/O **throughput**
- ❑ The maximum bus speed is largely limited by
  - ❑ The **length** of the bus
  - ❑ The **number** of devices on the bus

# Types of Buses

---

- ❑ Processor-memory bus (“Front Side Bus”, proprietary)
  - ❑ Short and high speed
  - ❑ Matched to the memory system to maximize the memory-processor bandwidth
  - ❑ Optimized for cache block transfers
- ❑ I/O bus (industry standard, e.g., SCSI, USB, Firewire)
  - ❑ Usually is lengthy and slower
  - ❑ Needs to accommodate a wide range of I/O devices
  - ❑ Use either the processor-memory bus or a backplane bus to connect to memory
- ❑ Backplane bus (industry standard, e.g., ATA, PCIeexpress)
  - ❑ Allow processor, memory and I/O devices to coexist on a single bus
  - ❑ Used as an intermediary bus connecting I/O busses to the processor-memory bus

# I/O Transactions

- ❑ An I/O transaction is a sequence of operations over the interconnect that includes a request and may include a response either of which may carry data.

- ❑ An I/O transaction typically includes two parts

1. Sending the address
2. Receiving or sending the data

- ❑ Bus transactions are defined by what they do to memory

output ❑ A **read** transaction reads data from memory (to either the processor or an I/O device)

input ❑ A **write** transaction writes data to the memory (from either the processor or an I/O device)

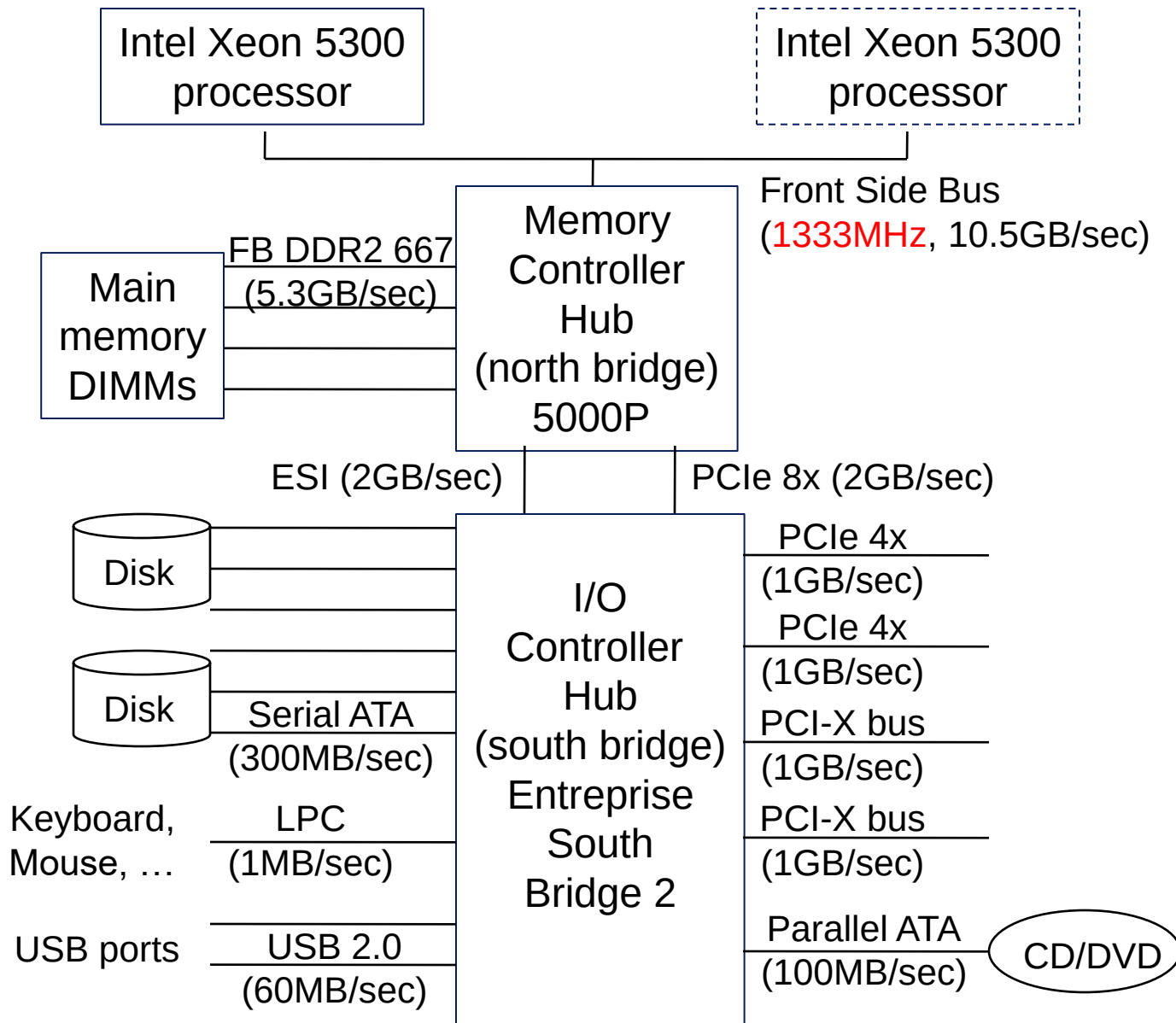
# Synchronous and Asynchronous Buses

- ❑ Synchronous bus (e.g., processor-memory buses)
  - ❑ Includes a clock in the control lines and has a fixed protocol for communication that is **relative** to the clock
  - ❑ Advantage: involves very little logic and can run very fast
  - ❑ Disadvantages:
    - Every device communicating on the bus must use same clock rate
    - Short distance
- ❑ Asynchronous bus (e.g., I/O buses)
  - ❑ It is not clocked, so requires a handshaking protocol and additional control lines (ReadReq, Ack, DataRdy)
  - ❑ Advantages:
    - Can accommodate a wide range of devices and device speeds
    - Can be lengthened without worrying about clock skew or synchronization problems
  - ❑ Disadvantage: slow(er)

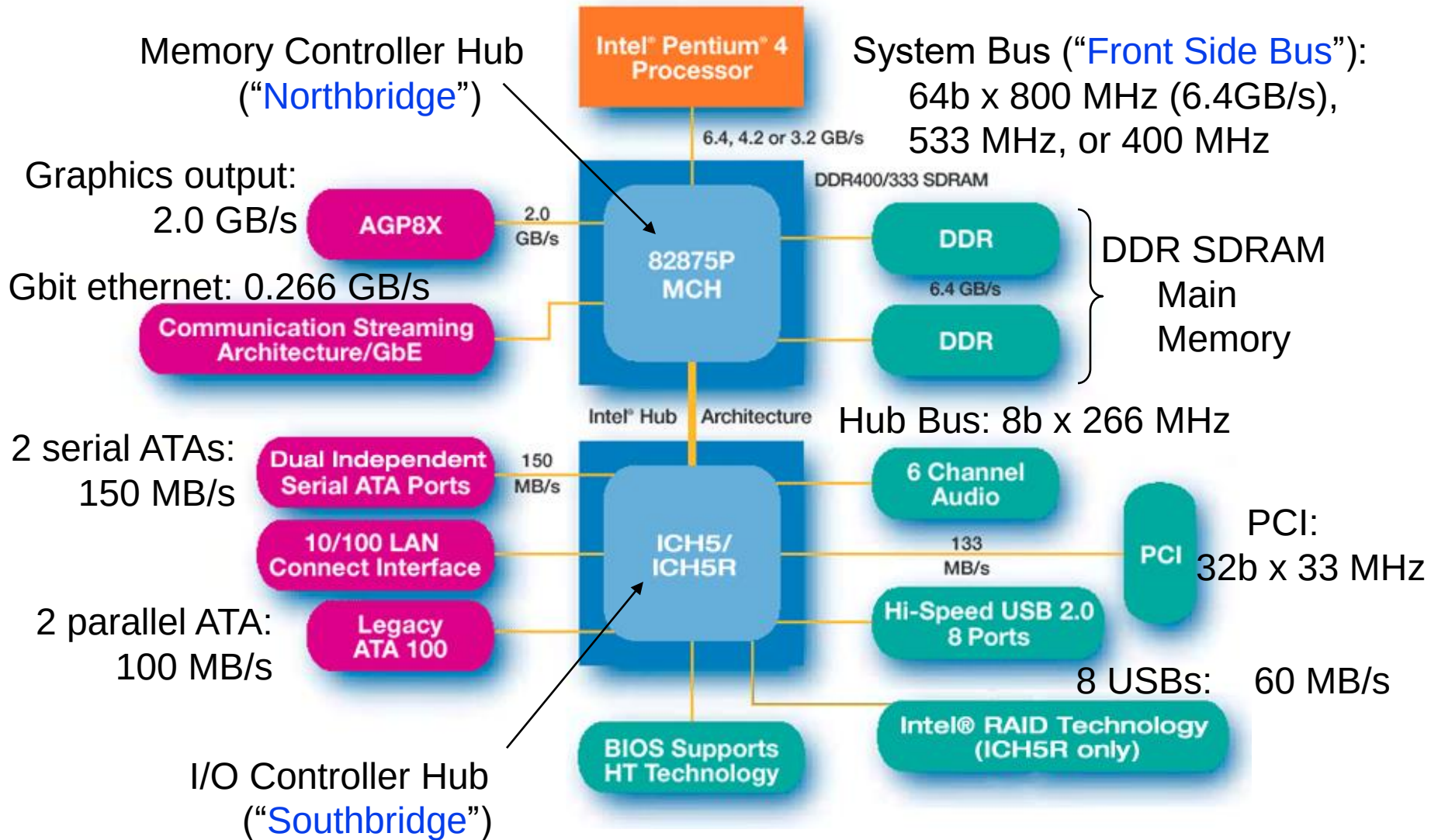
# Key Characteristics of some I/O Standards

|                     | <b>Firewire</b>                   | <b>USB 2.0</b>                                         | <b>PCIe</b>                                                 | <b>Serial ATA</b> | <b>SA SCSI</b> |
|---------------------|-----------------------------------|--------------------------------------------------------|-------------------------------------------------------------|-------------------|----------------|
| Use                 | External                          | External                                               | Internal                                                    | Internal          | External       |
| Devices per channel | 63                                | 127                                                    | 1                                                           | 1                 | 4              |
| Max length          | 4.5 meters                        | 5 meters                                               | 0.5 meters                                                  | 1 meter           | 8 meters       |
| Data Width          | 4                                 | 2                                                      | 2 per lane                                                  | 4                 | 4              |
| Peak Bandwidth      | 50MB/sec (400)<br>100MB/sec (800) | 0.2MB/sec (low)<br>1.5MB/sec (full)<br>60MB/sec (high) | 250MB/sec per lane (1x)<br>Come as 1x, 2x, 4x, 8x, 16x, 32x | 300MB/sec         | 300MB/sec      |
| Hot pluggable?      | Yes                               | Yes                                                    | Depends                                                     | Yes               | Yes            |

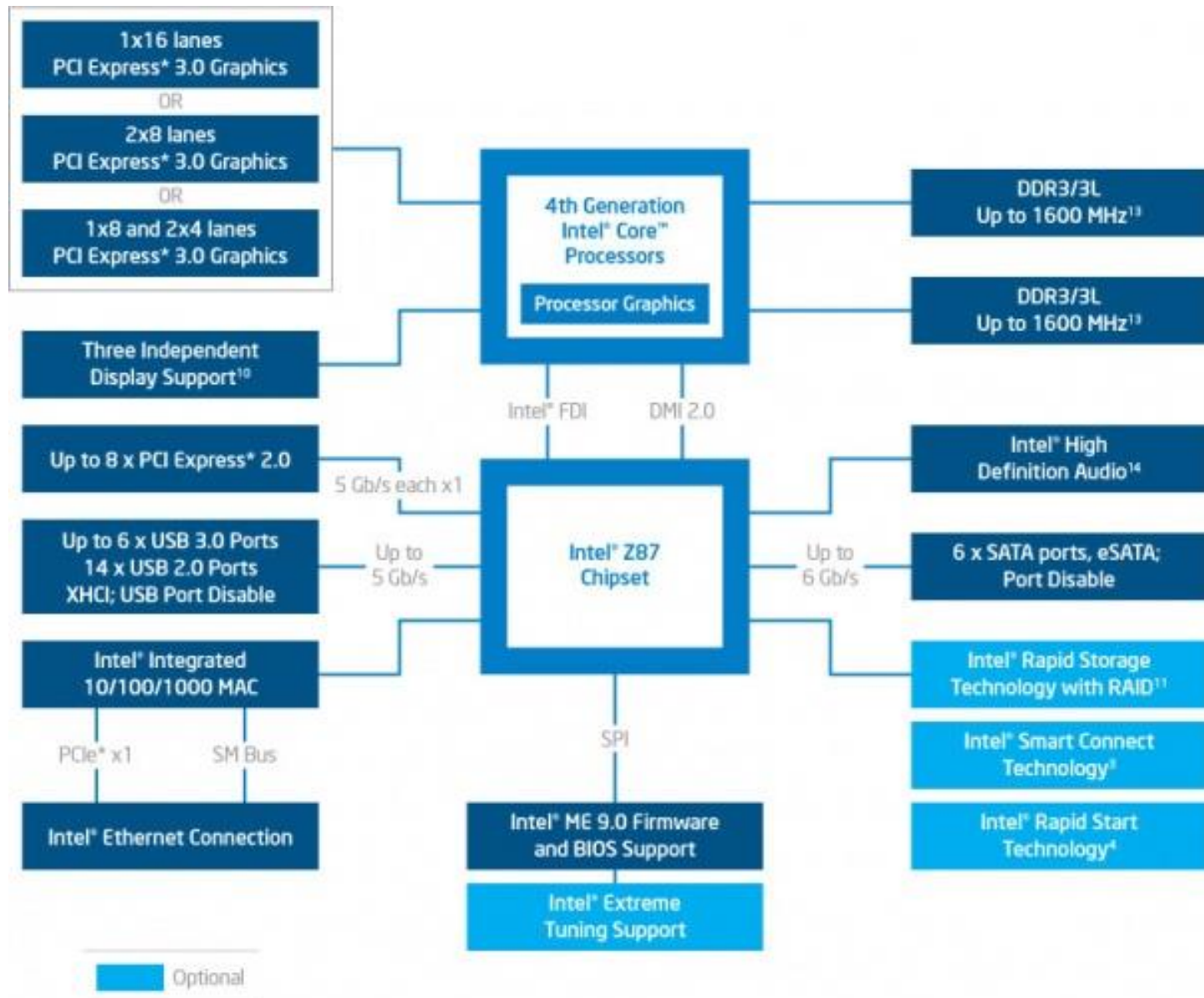
# A Typical I/O System



# Example: The Pentium 4's Buses



# Intel Core i7 with Z87 chipset



# Interfacing I/O Devices

---

- ❑ Physical connection is done, now how about data transfer?
- ❑ How is a user I/O request transformed into a device command and communicated to the device?
- ❑ How is data actually transferred to or from a memory location?
- ❑ What is the role of the operating system?

# Communication of I/O Devices and Processor

- ❑ How the processor directs (find) the I/O devices
  - ❑ Special I/O instructions
    - Must specify both the device and the command
  - ❑ Memory-mapped I/O
    - I/O devices are mapped to memory addresses
    - Read and writes to those memory addresses are interpreted as commands to the I/O devices
    - Load/stores to the I/O address space can *only* be done by the OS
- ❑ How I/O devices communicate with the processor
  - ❑ Polling – the processor periodically checks the status of an I/O device to determine its need for service
    - Processor is totally in control – but does **all** the work
    - Can waste a lot of processor time due to speed differences
  - ❑ Interrupt-driven I/O – the I/O device issues an interrupt to indicate that it needs attention

# Interrupt Driven I/O

---

- ❑ An I/O interrupt is **asynchronous** instruction execution
  - ❑ Is not associated with any instruction so doesn't prevent any instruction from completing
    - You can pick your own convenient point to handle the interrupt
- ❑ With I/O interrupts
  - ❑ Need a way to identify the device generating the interrupt
  - ❑ Can have different urgencies (so need a way to **prioritize** them)
- ❑ Advantages of using interrupts
  - ❑ Relieves the processor from having to continuously poll for an I/O event; user program progress is only suspended during the actual transfer of I/O data to/from user memory space
- ❑ Disadvantage – special hardware is needed to
  - ❑ Indicate the I/O device causing the interrupt and to save the necessary information prior to servicing the interrupt and to resume normal processing after servicing the interrupt

# Direct Memory Access (DMA)

---

- ❑ For high-bandwidth devices (like disks) interrupt-driven I/O would consume a *lot* of processor cycles
- ❑ With DMA, the DMA controller has the ability to transfer large blocks of data **directly** to/from the memory without involving the processor
  1. The processor initiates the DMA transfer by supplying the I/O device address, the operation to be performed, the memory address destination/source, the number of bytes to transfer
  2. The DMA controller manages the entire transfer (possibly thousand of bytes in length), arbitrating for the bus
  3. When the DMA transfer is complete, the DMA controller interrupts the processor to let it know that the transfer is complete
- ❑ There may be multiple DMA devices in one system
  - ❑ Processor and DMA controllers contend for bus cycles and for memory

# Summary

---

- ❑ Characteristics of I/O system and devices
- ❑ I/O performance measures
- ❑ I/O system organization
- ❑ Methods for I/O operation and control
  - ❑ Polling
  - ❑ Interrupt
  - ❑ DMA